

Experiments were run on the following environments: **PointMaze-umaze** and **PointMaze-medium**, **PushT**, and **Wall**. All train decoder-less world models and evaluate open and closed loop planning. All evals were single seed.

Power Law Regularizer (P-Reg)

The idea is to reduce the speed of trajectories in places with high curvature. If the spacing of latent trajectories that the planner needs to reach is proportional to the amount of curvature it could help with planning.

This comes naturally from the two-thirds power law which states that the instantaneous speed of a biological movement and the curvature of its path are coupled by an empirical invariant. I use this as motivation behind my regularizer (P-Reg)

Implementation

- from z_{t-1} , z_t and z_{t+1} we get v_1 and v_2 .
- Speed $s = (||v_1|| + ||v_2||) / 2$
- $\theta = \text{angle bw } (v_1, v_2)$
- $k = \theta / s \rightarrow \# \text{ curvature}$

It re-uses quantities already calculated by the straightening regularizer.

- $r = \log s + 1/3 \log k$

The term r couples speed and curvature. We want this term to remain constant so that an increase in curvature results in lower speeds along the trajectory window.

Then I regularize its variance along the trajectory rather than its value (this only constrains the coupling exponent, not the absolute speed or curvature, so it does not compete with L_{pred} for absolute scale).

- $L_{\text{PReg}} = \text{Var}(r) \text{ over } t$

In my experiments $\text{num_hist}=3$ and $\text{num_pred}=1$. So there are only two residuals calculated (Very local). This is done for compute reasons. Could change window size later.

- (L_{PReg} is just squared distance between r_1 and r_0 currently.)
- Temporal straightening takes mean of 2 curvatures in my experiments

Closed-Loop MPC Results

Each eval episode is executed in the real sim and re-planned until success or the iteration cap. Per env, the four variants are **baseline**, **straighten**, **P-Reg**, and **P-Reg + straighten** (the "both" model).

MPC settings

- **n_evals=50** - 50 executed episodes, **goal_H=25**, **seed=100** (eval on single seed)
- MPC loop: **n_taken_actions=5** (1 latent with frameskip = 5), **max_iter=4** (safety cap; exits early on success)

- **GD-MPC**: sub-planner GD, **opt_steps=100**, Adam **lr=0.1**, zero init
- Objectives: umaze/medium **objective.alpha=0 mode=all** (weighted intermediate-state loss);
pusht GD-MPC **alpha=1 mode=staged**; pusht CEM **alpha=1 mode=last**

PushT

Variant	GD success_rate	GD state- dist	GD visual- dist	GD proprio- dist	CEM success_rate	CEM state- dist
baseline	0.720	118.1	3.164	42.42	—	—
straighten	0.860	70.3	2.681	24.76	—	—
p-reg	0.840	57.2	3.007	20.88	—	—
p- reg+straighten	0.920	53.8	2.707	19.22	—	—

- Planning cost (total MPC iters / 50 eps): GD 520/413/432/378 (baseline / straighten / p-reg / both).

PointMaze-umaze

Variant	GD success_rate	GD state- dist	GD visual- dist	GD proprio- dist	CEM success_rate	CEM state- dist
baseline	0.700	3.220	0.490	1.191	—	—
straighten	0.880	3.892	0.460	1.325	—	—
p-reg	0.660	3.867	0.471	1.427	—	—
p-reg +straighten	0.900	3.756	0.460	1.268	—	—

- Planning cost (total MPC iters / 50 eps): GD 541/366/549/311 (baseline / straighten / p-reg / both) — fewer iterations means earlier success on average.

PointMaze-medium

Variant	GD success_rate	GD state- dist	GD visual- dist	GD proprio- dist	CEM success_rate	CEM state- dist
baseline	0.620	4.132	0.301	1.550	—	—
straighten	0.700	4.814	0.290	1.699	—	—
p-reg	0.600	4.385	0.306	1.736	—	—

Variant	GD success_rate	GD state- dist	GD visual- dist	GD proprio- dist	CEM success_rate	CEM state- dist
p-reg +straighten	0.760	4.309	0.290	1.513	—	—

- Planning cost (total MPC iters / 50 eps): GD 559/491/542/528 (baseline / straighten / p-reg / both).

Open-Loop MPC Results

(`max_iter=1`).

PushT results (encoder = dino_channel)

All four models trained at: `batch_size=16`, `epochs=2`, `frameskip=5`, `num_hist=3`, `TRAIN_DECODER=False`.

Model (dino_channel)	GD success	CEM success	GD state-dist	CEM state-dist
baseline	0.74	0.72	50.62	50.82
straightening only	0.80	0.78	51.41	53.06
p-reg only	0.54	0.50	41.66	51.78
both	0.80	0.80	37.73	45.41

PointMaze (umaze) results

Model (dino_global)	GD success	CEM success	GD state-dist	CEM state-dist
baseline	0.18	0.56	3.54	2.73
straightening only	0.44	0.52	2.36	2.76
p-reg only	0.20	0.58	3.68	2.97
both	0.60	0.62	2.33	2.82

- Planning is open-loop (`MPC max_iter=1`): one action sequence is planned in latent space (5 latent steps x 5 frameskip = 25 env frames) and executed.

PointMaze (medium) results

Same protocol as umaze (`dino_global`, decoder-less, 20 epochs, open-loop GD/CEM at `goal_H=25`) but on the D4RL maze2d-**medium** layout (4000 x 100-step episodes).

Model (dino_global)	GD success	CEM success	GD state-dist	CEM state-dist
baseline	0.08	0.38	4.31	3.26
straightening only	0.12	0.46	3.69	3.36
p-reg only	0.22	0.24	3.85	3.70
both	0.16	0.46	4.03	3.19

Wall results (encoder = dino_channel)

Wall uses the paper's setup: start/goal states are sampled from **test trajectories** (`goal_source='dset'`, the `plan_gd/plan_cem` default) so every goal is reachable within 25 steps.

Model (dino_channel)	GD success	CEM success	GD state-dist	CEM state-dist
baseline	0.36	0.44	6.56	5.54
straightening only	0.90	1.00	1.99	1.52
p-reg only	0.68	0.68	5.29	4.55
both	0.68	0.60	3.52	3.68